

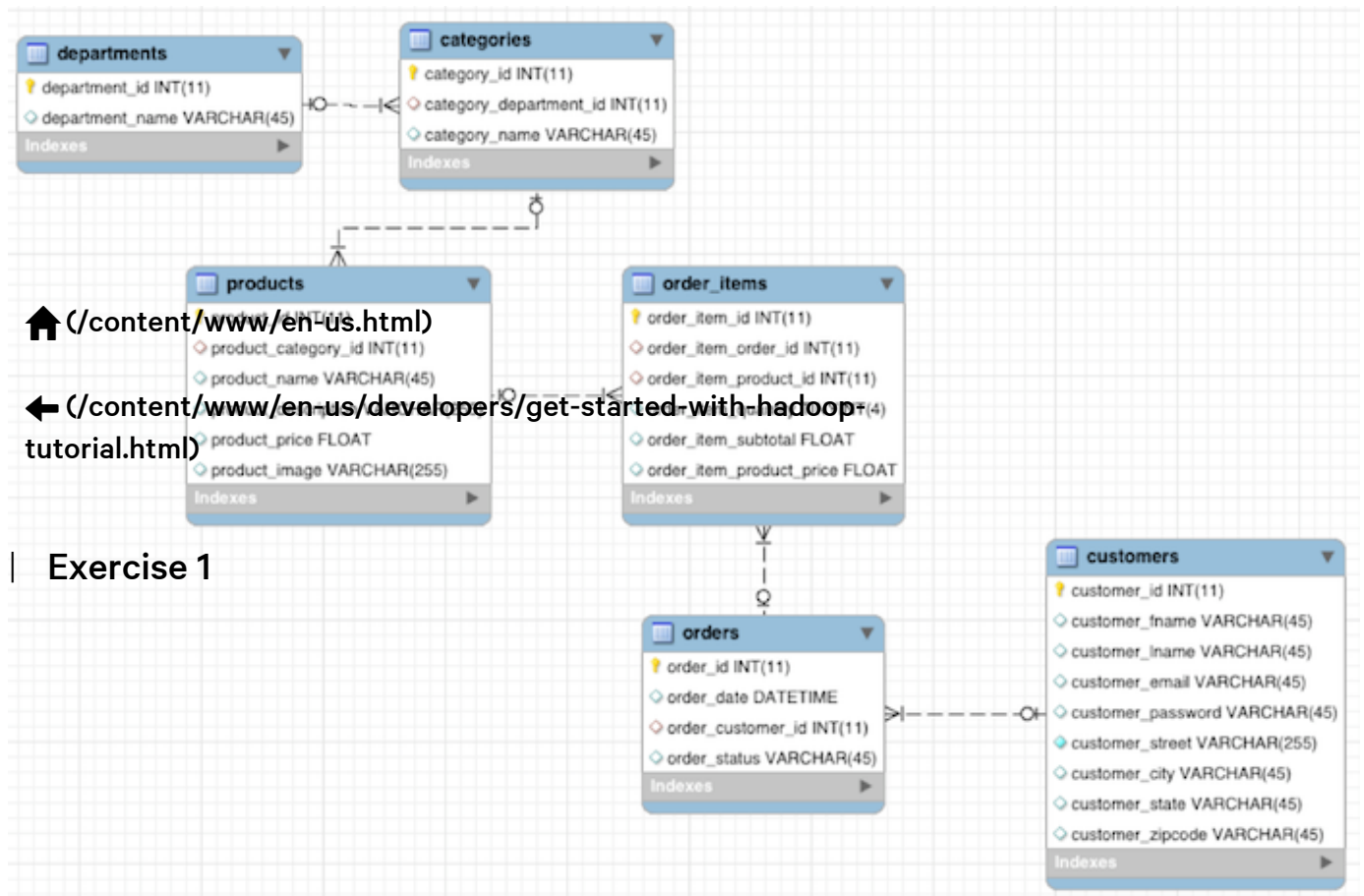
# The "Getting Started with Hadoop" Tutorial

## Exercise 1: Ingest and query relational data

In this scenario, DataCo's business question is: What products do our customers like to buy? To answer this question, the first thought might be to look at the transaction data, which should indicate what customers actually do buy and like to buy, right?

This is probably something you can do in your regular RDBMS environment, but a benefit of Apache Hadoop is that you can do it at greater scale at lower cost, on the same system that you may also use for many other types of analysis.

What this exercise demonstrates is how to do exactly the same thing you already know how to do, but in CDH. Seamless integration is important when evaluating any new infrastructure. Hence, it's important to be able to do what you normally do, and not break any regular BI reports or workloads over the dataset you plan to migrate.



## Exercise 1

To analyze the transaction data in the new platform, we need to ingest it into the Hadoop Distributed File System (HDFS). We need to find a tool that easily transfers structured data from a RDBMS to HDFS, while preserving structure. That enables us to query the data, but not interfere with or break any regular workload on it.

Apache Sqoop, which is part of CDH, is that tool. The nice thing about Sqoop is that we can automatically load our relational data from MySQL into HDFS, while preserving the structure. With a few additional configuration parameters, we can take this one step further and load this relational data directly into a form ready to be queried by Impala (the open source analytic query engine included with CDH) and other workloads.

You should first log in to the Master Node of your cluster -- using SSH if using **Cloudera Live** (</content/www/en-us/get-started/cloudera-live.html>) (see the instructions on the "Your Cloudera Cluster" page), or via a terminal if using a different method. Then, launch the Sqoop job:

```
> sqoop import-all-tables \  
  -m {{cluster_data.worker_node_hostname.length}} \  
  --connect jdbc:mysql://{{cluster_data.manager_node_hostname}} \  
  --username=retail_dba \  
  --password=cloudera \  
  --compression-codec=snappy \  
  --as-parquetfile \  
  --warehouse-dir=/user/hive/warehouse \  
  --hive-import
```

This command may take a while to complete, but it is doing a lot. It is launching MapReduce jobs to pull the data from our MySQL database and write the data to HDFS in parallel, distributed across the cluster in **Apache Parquet** (<http://parquet.apache.org>) format. It is also creating tables to represent the HDFS files in Impala/Apache Hive with matching schema.

Parquet is a format designed for analytical applications on Hadoop. Instead of grouping your data into rows like typical data formats, it groups your data into columns. This is ideal for many analytical queries where instead of retrieving data from specific records, you're analyzing relationships between specific variables across many records. Parquet is designed to optimize data storage and retrieval in these scenarios.

## Verification

When this command is complete, confirm that your data files exist in HDFS.

```
> hadoop fs -ls /user/hive/warehouse/  
> hadoop fs -ls /user/hive/warehouse/categories/
```

These commands will show the directories and the files inside them that make up your tables:

```

FILE: Number of large read operations=0
FILE: Number of write operations=0
HDFS: Number of bytes read=24930
HDFS: Number of bytes written=62966
HDFS: Number of read operations=90
HDFS: Number of large read operations=0
HDFS: Number of write operations=27
Job Counters
  Launched map tasks=3
  Other local map tasks=3
  Total time spent by all maps in occupied slots (ms)=27356
  Total time spent by all reduces in occupied slots (ms)=0
  Total time spent by all map tasks (ms)=27356
  Total vcore-seconds taken by all map tasks=27356
  Total megabyte-seconds taken by all map tasks=28012544
Map-Reduce Framework
  Map input records=1345
  Map output records=1345
  Input split bytes=354
  Spilled Records=0
  Failed Shuffles=0
  Merged Map outputs=0
  GC time elapsed (ms)=281
  CPU time spent (ms)=13110
  Physical memory (bytes) snapshot=1044811776
  Virtual memory (bytes) snapshot=4748353536
  Total committed heap usage (bytes)=1308622848
File Input Format Counters
  Bytes Read=0
File Output Format Counters
  Bytes Written=0
15/06/22 14:32:39 INFO mapreduce.ImportJobBase: Transferred 61.4902 KB in 30.9012 seconds (1.9899 KB/sec)
15/06/22 14:32:39 INFO mapreduce.ImportJobBase: Retrieved 1345 records.
[root@cloudera1 ~]# hadoop fs -ls /user/hive/warehouse/
Found 6 items
drwxrwxrwt - root hive 0 2015-06-22 14:29 /user/hive/warehouse/categories
drwxrwxrwt - root hive 0 2015-06-22 14:30 /user/hive/warehouse/customers
drwxrwxrwt - root hive 0 2015-06-22 14:30 /user/hive/warehouse/departments
drwxrwxrwt - root hive 0 2015-06-22 14:31 /user/hive/warehouse/order_items
drwxrwxrwt - root hive 0 2015-06-22 14:32 /user/hive/warehouse/orders
drwxrwxrwt - root hive 0 2015-06-22 14:32 /user/hive/warehouse/products
[root@cloudera1 ~]#
[root@cloudera1 ~]# hadoop fs -ls /user/hive/warehouse/categories/
Found 4 items
drwxr-xr-x - root hive 0 2015-06-22 14:29 /user/hive/warehouse/categories/.metadata
-rw-r--r-- 2 root supergroup 1295 2015-06-22 14:29 /user/hive/warehouse/categories/392c0770-0eb5-41b4-8137-0470641d0d14.parquet
-rw-r--r-- 2 root supergroup 1281 2015-06-22 14:29 /user/hive/warehouse/categories/92b8ec8c-9bfb-4256-9f4f-8d86e955fae1.parquet
-rw-r--r-- 2 root supergroup 1334 2015-06-22 14:29 /user/hive/warehouse/categories/df614aba-db6d-44a8-8364-6b4cd506753a.parquet
[root@cloudera1 ~]#
[root@cloudera1 ~]#

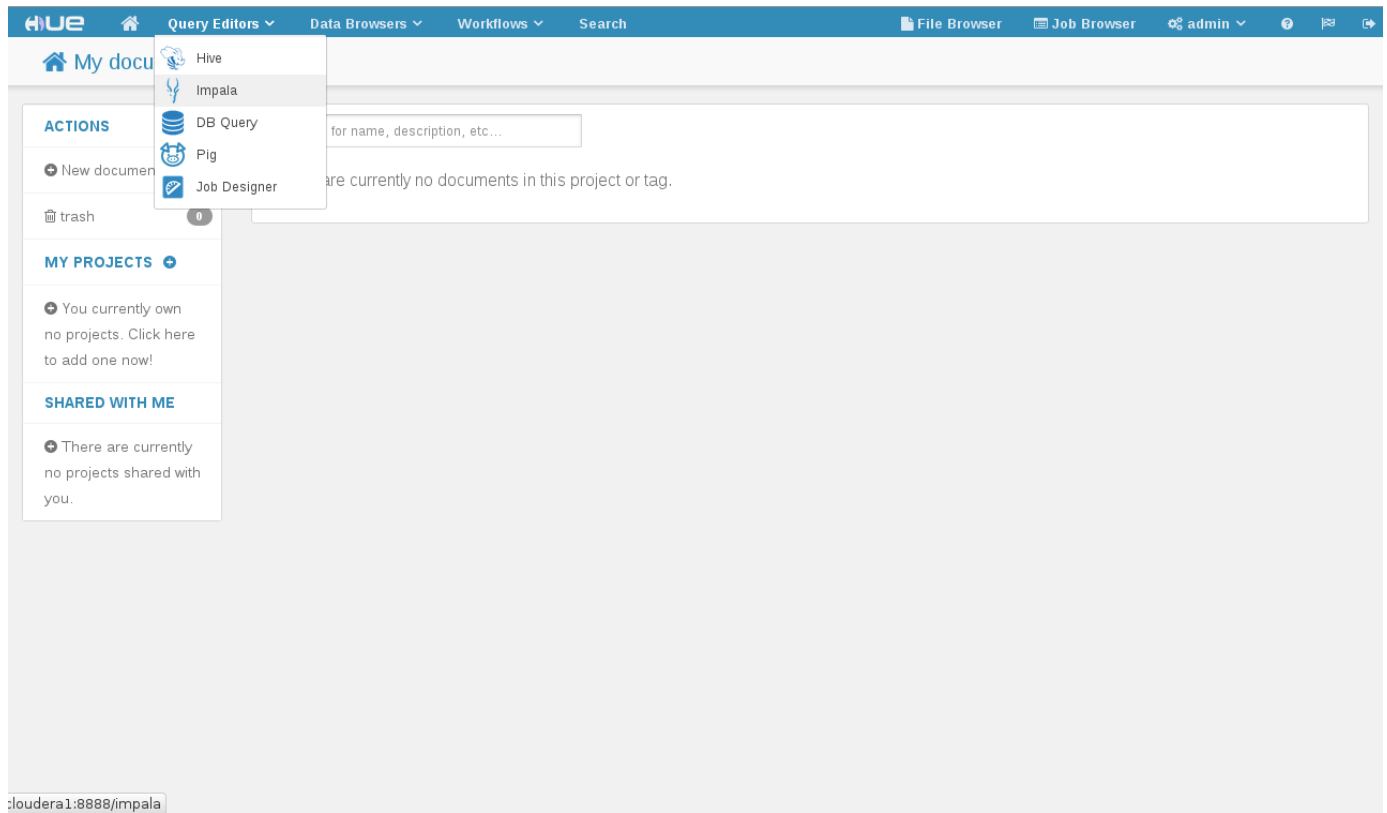
```

**Note:** The number of .parquet files shown will be equal to what was passed to Sqoop with the -m parameter. This is the number of 'mappers' that Sqoop will use in its MapReduce jobs. It could also be thought of as the number of simultaneous connections to your database, or the number of disks / Data Nodes you want to spread the data across. So on a single-node you will just see one, but larger clusters will have a greater number of files.

Hive and Impala also allow you to create tables by defining a schema over existing files with 'CREATE EXTERNAL TABLE' statements, similar to traditional relational databases. But Sqoop already created these tables for us, so we can go ahead and query them.

We're going to use Hue's Impala app to query our tables. Hue provides a web-based interface for many of the tools in CDH and can be found on port 8888 of your Manager Node. If using Cloudera Live, refer to your welcome email to find your credentials for logging into Hue. In the QuickStart VM, the administrator username for Hue is 'cloudera' and the password is 'cloudera'.

Once you are inside of Hue, click on Query Editors, and open the Impala Query Editor.



To save time during queries, Impala does not poll constantly for metadata changes. So the first thing we must do is tell Impala that its metadata is out of date. Then we should see our tables show up, ready to be queried:

```
invalidate metadata;  
  
show tables;
```

You can also click on the "Refresh Table List" icon on the left to see your new tables in the side menu.

The screenshot shows the Hue Query Editor interface. The top navigation bar includes 'Query Editors', 'Data Browsers', 'Workflows', 'Search', 'File Browser', 'Job Browser', and 'admin'. The 'Query Editor' tab is active, showing 'My Queries', 'Saved Queries', and 'History'. On the left, the 'DATABASE' dropdown is set to 'default', and a list of tables is visible: categories, customers, departments, order\_items, orders, and products. The main query editor contains the following SQL code:

```
1 invalidate metadata;
2
3 show tables;
```

Below the query editor are buttons for 'Execute', 'Save as...', 'Explain', and 'New query'. The 'Execute' button has been clicked, and the results are displayed in a table with the following data:

|   | name        |
|---|-------------|
| 0 | categories  |
| 1 | customers   |
| 2 | departments |
| 3 | order_items |
| 4 | orders      |
| 5 | products    |

The status bar at the bottom indicates the query was executed successfully: 'cloudera1:8888/impala/execute/query/8#'.

Now that your transaction data is readily available for structured queries in CDH, it's time to address DataCo's business question. Copy and paste or type in the following standard SQL example queries for calculating total revenue per product and showing the top 10 revenue generating products:

```
-- Most popular product categories
select c.category_name, count(order_item_quantity) as count
from order_items oi
inner join products p on oi.order_item_product_id = p.product_id
inner join categories c on c.category_id = p.product_category_id
group by c.category_name
order by count desc
limit 10;
```

You should see results of the following form:

The screenshot shows the Hue Impala Query Editor interface. On the left, there's a sidebar with 'Assist' and 'Settings' tabs, a 'DATABASE' dropdown set to 'default', and a list of tables: categories, customers, departments, order\_items, orders, and products. The main area displays a SQL query:

```

1 -- Most popular product categories
2 select c.category_name, count(order_item_quantity) as count
3 from order_items oi
4 inner join products p on oi.order_item_product_id = p.product_id
5 inner join categories c on c.category_id = p.product_category_id
6 group by c.category_name
7 order by count desc
8 limit 10;

```

Below the query are buttons: 'Execute', 'Save as...', 'Explain', 'or create a', and 'New query'. The 'Results' tab is active, showing a table with 10 rows:

|   | category_name        | count |
|---|----------------------|-------|
| 0 | Men's Footwear       | 23029 |
| 1 | Cleats               | 19786 |
| 2 | Women's Apparel      | 19685 |
| 3 | Indoor/Outdoor Games | 18456 |
| 4 | Fishing              | 16991 |
| 5 | Water Sports         | 15517 |
| 6 | Camping & Hiking     | 13955 |
| 7 | Cardio Equipment     | 12827 |
| 8 | Shop By Sport        | 11446 |
| 9 | Electronics          | 3694  |

Clear out the previous query, and replace it with the following:

```

-- top 10 revenue generating products
select p.product_id, p.product_name, r.revenue
from products p inner join
(select oi.order_item_product_id, sum(cast(oi.order_item_s
from order_items oi inner join orders o
on oi.order_item_order_id = o.order_id
where o.order_status <> 'CANCELED'
and o.order_status <> 'SUSPECTED_FRAUD'
group by order_item_product_id) r
on p.product_id = r.order_item_product_id
order by r.revenue desc
limit 10;

```

You should see results similar to this:

The screenshot shows the Hue Impala Query Editor interface. The top navigation bar includes links for Query Editors, Data Browsers, Workflows, Search, File Browser, Job Browser, and an admin dropdown. The main header shows 'Impala Query Editor' with tabs for My Queries, Saved Queries, and History. On the left, there's a sidebar with 'Assist' and 'Settings' tabs, a 'DATABASE' dropdown set to 'default', and a list of tables including categories, customers, departments, order\_items, orders, and products. The main area contains a SQL query editor with the following code:

```

1 -- top 10 revenue generating products
2 select p.product_id, p.product_name, r.revenue
3 from products p inner join
4 (select oi.order_item_product_id, sum(cast(oi.order_item_subtotal as float)) as revenue
5 from order_items oi inner join orders o
6 on oi.order_item_order_id = o.order_id
7 where o.order_status <> 'CANCELED'
8 and o.order_status <> 'SUSPECTED_FRAUD'
9 group by order_item_product_id) r
10 on p.product_id = r.order_item_product_id
11 order by r.revenue desc
12 limit 10;

```

Below the query editor are buttons for 'Execute', 'Save as...', 'Explain', and 'New query'. The results tab is active, showing a table with 9 rows and 4 columns: product\_id, product\_name, and revenue. The data is as follows:

|   | product_id | product_name                                  | revenue            |
|---|------------|-----------------------------------------------|--------------------|
| 0 | 1004       | Field & Stream Sportsman 16 Gun Fire Safe     | 6481676.0780334473 |
| 1 | 957        | Diamondback Women's Serene Classic Comfort Bi | 4002333.3065795898 |
| 2 | 191        | Nike Men's Free 5.0+ Running Shoe             | 3595240.4369888306 |
| 3 | 365        | Perfect Fitness Perfect Rip Deck              | 3396753.8754653931 |
| 4 | 1073       | Pelican Sunstream 100 Kayak                   | 2958452.151260376  |
| 5 | 403        | Nike Men's CJ Elite 2 TD Football Cleat       | 2859780.1208496094 |
| 6 | 502        | Nike Men's Dri-FIT Victory Golf Polo          | 2819750            |
| 7 | 1014       | O'Brien Men's Neoprene Life Vest              | 2651588.9155731201 |
| 8 | 627        | Under Armour Girls' Toddler Spine Surge Runni | 1265723.51121521   |

You may notice that we told Sqoop to import the data into Hive but used Impala to query the data. This is because Hive and Impala can share both data files and the table metadata. Hive works by compiling SQL queries into MapReduce jobs, which makes it very flexible, whereas Impala executes queries itself and is built from the ground up to be as fast as possible, which makes it better for interactive analysis. We'll use Hive later for an ETL (extract-transform-load) workload.

## Conclusion:

Now that you have gone through the first basic steps to Sqoop structured data into HDFS, transform it into Parquet file format, and create hive tables for use when you query this data.

You have also learned how to query tables using Impala and that you can use regular interfaces and tools (such as SQL) within a Hadoop environment as well. The idea here being that you can do the same reports you usually do, but where the architecture of Hadoop vs traditional systems provides much larger scale and flexibility.

**GO TO NEXT STEP** (</content/www/en-us/developers/get-started-with-hadoop-tutorial/showing-big-data-value.html>)

**About Cloudera** (</content/www/en-us/about-cloudera.html>)



**Resources (/content/www/en-us/resources.html)**

**Contact (/content/www/en-us/contact-us.html)**

**Careers (/content/www/en-us/about-cloudera/careers.html)**

**Press (/content/www/en-us/about-cloudera/press-center.html)**

**Documentation (/content/www/en-us/documentation.html)**

United States: +1 888 789 1488

Outside the US: +1 650 362 0488

(<https://www.linkedin.com/company/cloudera>)

(<https://www.facebook.com/cloudera>)

(<https://twitter.com/cloudera>)

(</content/www/en-us/contact-us.html>)

**Terms & Conditions (/content/www/en-us/legal/terms-and-conditions.html) |**

**Privacy Policy (/content/www/en-us/legal/privacy-policy.html)**